HRMS Platform

Project Analysis and Technical Documentation

Source-grounded review of the application architecture, implemented modules, APIs, data model, security, user flows, and operations.

Prepared from the repository working tree
11 November 2026

This report compiles the project documentation package. Implementation gaps and items needing owner confirmation are called out where evidence is incomplete.

Contents

01 Project Overview
02 Software Requirements Specification
03 System Architecture
04 Module Wise Documentation
05 Database Design
06 API Documentation
07 Authentication and Authorization
08 User Workflows
09 UI UX Documentation
10 Installation and Setup
11 Testing Documentation
12 Deployment and Maintenance
13 Project Structure
14 Documentation Index
15 Project UI Examples

Project Overview

Purpose

HRMS Platform is a browser-based, multi-tenant human resources application. It combines a React single-page application with a Spring Boot REST service and PostgreSQL persistence. Implemented areas span employee records, attendance, leave, payroll, recruiting, approvals, documents, expenses, performance, shift rosters, support tickets and tenant administration.

This description reflects the repository implementation, not a claim of regulatory, payroll-provider, or enterprise readiness. See module status and review findings.

At a glance

Concern	Implementation
Backend	Java 21 target, Spring Boot 3.2.3, Spring MVC, Spring Security, Spring Data JPA
Frontend	React 18, TypeScript/JavaScript, Vite, React Router, Redux Toolkit, Axios, Tailwind CSS
Persistence	PostgreSQL-oriented configuration, JPA entities, Flyway migrations V1–V15
Authentication	Bearer JWT access tokens, refresh tokens, BCrypt, optional TOTP endpoints
Tenancy	Request tenant context filter; tenant-aware entities/repositories and tenant administration
Operations	Docker Compose for PostgreSQL, Redis, backend, frontend, Prometheus and Grafana
API prefix	`/api/v1` under the default `server.servlet.context-path=/api`; controller mappings themselves begin `/v1`

Implemented business areas

Core functionality is organized around employee and department records, attendance and attendance policies, leave and balances, employee requests and approval workflows, payroll, recruitment and interview sessions, performance, assets, expenses, documents and policy acknowledgements, offboarding, helpdesk, announcements, notifications, shifts, reporting, workforce intelligence, tenant branding/modules, and administrative audit/settings. Evidence and caveats are summarized in Module Wise Documentation.

Architecture summary

The UI calls REST APIs through shared Axios clients and module-level API/service wrappers. Spring controllers validate and route requests to services; services coordinate repositories and domain rules; JPA entities persist to tenant-scoped tables. Flyway provides schema evolution. JWT and tenant filters run in the request chain. Email uses Spring Mail/Thymeleaf; Redis is used for distributed token blacklisting where configured; live interview functionality includes signaling/token components and a frontend LiveKit client. See System Architecture.

Boundaries and open questions

The repository demonstrates a broad feature surface, but production topology, actual tenant isolation under all service queries, external identity integration, payroll statutory correctness, backup/restore procedures, and SLA behavior require operational confirmation. AI/assistant endpoints are present, but no external model provider integration is established by the source reviewed; the service implementation should be reviewed before describing responses as generative AI.

Software Requirements Specification

1. Purpose and scope

This SRS records observable requirements implied by the current source code. It is not a signed product specification. Requirement identifiers below are descriptive and do not imply contractual acceptance.

2. User classes

User class	Evidence in code	Expected access pattern
Employee	`Role.ROLE_EMPLOYEE`, employee UI, self-service endpoints	Profile, attendance, leave, requests, documents, notifications, expenses and related self-service features
Manager	`ROLE_MANAGER`, approval and operational endpoints	Review workflows, attendance/shift requests, team operations where implemented
HR	`ROLE_HR`	Employee, leave, attendance, payroll, recruiting and policy administration according to endpoint guards
Tenant administrator	`ROLE_ADMIN`	Tenant-level administration, configuration and operational controls
Super administrator	`ROLE_SUPER_ADMIN`	Cross-tenant administration and selected elevated operations
IT roles	`ROLE_IT_ADMIN`, `ROLE_IT_SUPPORT`	Selected admin, asset and helpdesk operations

Role names are not a complete permission matrix; route and method guards vary. See authorization.

3. Functional requirements

ID	Requirement supported by code	Principal evidence
FR-01	Users can register/login, verify email, refresh/logout, change/reset passwords, review login history and configure TOTP.	`AuthController`, `AuthService`, `TotpService`
FR-02	Authorized users can create, search, view, update and soft-delete employee and department records; employees can access profile-related surfaces.	`EmployeeController`, `DepartmentController`, services
FR-03	Attendance records can be recorded, queried and updated; punch, break, biometric, regularization and office-location paths are exposed.	`AttendanceController`, `AttendanceService`, migrations V3/V6/V15
FR-04	Leave applications, reviews, cancellation, leave policies, balances and accrual operations are exposed.	`LeaveController`, `LeavePolicyController`, `LeaveService`, schedulers
FR-05	Employee requests and configurable workflows can be submitted and reviewed; workflow definitions and approval delegations can be queried/managed.	`EmployeeRequestController`, `WorkflowController`, `WorkflowEngineService`
FR-06	Payroll can be processed, reviewed/adjusted, approved, marked paid, summarized, and exported for bank batch workflows.	`PayrollController`, `PayrollService`
FR-07	Job postings/candidates can be managed, candidate stages and scorecards updated, candidates converted; interviews support guest join and signaling routes.	Recruitment and interview controllers/services
FR-08	Performance cycles, employee goals, goal progress and reviews can be managed.	`PerformanceController`, `PerformanceService`
FR-09	Expenses can be submitted and reviewed; helpdesk tickets/comments, announcements and acknowledgements are supported.	Corresponding module controllers/services
FR-10	Documents can be uploaded, listed, verified, downloaded, acknowledged; HR letters can be generated.	`DocumentController`, `DocumentService`
FR-11	Offboarding/separation, clearance and settlement operations are exposed.	`OffboardingController`, `OffboardingService`
FR-12	Shift definitions, roster assignments, shift swaps and peer/manager review operations are exposed.	`ShiftRosterController`, `ShiftRosterService`
FR-13	Tenants can be inspected/configured and branded; tenant module manifests can be read and toggled.	Tenant/admin controllers and services
FR-14	Dashboard, reports, notifications, assets and workforce intelligence functions are available through APIs and UI routes.	Module controllers and frontend pages

4. Non-functional requirements visible in implementation

- **NFR-01 Security:** stateless Spring Security filter chain, JWT bearer parsing, BCrypt encoder, method-level authorization, CORS configuration, login/API rate limiting configuration and audit support are present. Configuration needs production review.

- **NFR-02 Tenant separation:** tenant ID is resolved from headers, query parameter, or host subdomain and stored in `TenantContext`; `BaseTenantEntity` and tenant-aware repositories support isolation. Verify every query and object ownership path.
- **NFR-03 Persistence:** JPA/Hibernate and Flyway are configured; PostgreSQL is the intended database for current profile/configuration.
- **NFR-04 Operability:** Spring Actuator, Micrometer Prometheus, Docker Compose and monitoring configuration exist.
- **NFR-05 Usability:** SPA routes, responsive utility classes, charts, file upload and error-boundary components are present. No accessibility conformance statement is verified.

5. Out of scope or not verified

No supported evidence establishes statutory compliance in any jurisdiction, integration with a real bank/biometric vendor, an external HRIS/identity provider, a production AI provider, guaranteed encryption at rest for file storage, formal disaster recovery objectives, or complete approval SLA enforcement. The code contains data structures and endpoints that may be prototypes or partial implementations.

6. Assumptions and questions

1. Confirm intended deployment database and migration history (V1 dialect concern).
2. Confirm which roles should own each self-service action and whether employee ID ownership is enforced.
3. Confirm payroll formulas and statutory assumptions with a qualified domain owner.
4. Confirm if the live interview signaling implementation is production-ready and which media provider is active.
5. Confirm retention, privacy, backup, audit and data residency requirements.

System Architecture

Architectural style

The system is a modular monolithic web application: a React SPA is deployed separately from a Spring Boot application. Backend code is organized by business module with controller/service/repository/entity/DTO layers. No independently deployable domain microservices were found.

Runtime components

Component	Responsibility	Evidence
React SPA	Routes, screens, state and API calls	`frontend/src/App.tsx`, `modules/`, `api/`, `store/`
REST API	HTTP endpoints and request validation	`backend/src/main/java/com/hrms/modules/*/controller`
Application services	Business orchestration and rules	module `service` packages
Persistence	ORM and repository access	JPA entities and Spring Data repositories
PostgreSQL	Relational data store	profiles, Flyway SQL, Compose
Redis	Token blacklist implementation	`DistributedTokenBlacklistService`, Compose
SMTP	Application email	`EmailServiceImpl`, Thymeleaf templates
Local storage	Profile/document uploads	`LocalStorageService`, `StorageService` port
Metrics	Health and Prometheus metrics	Actuator, Micrometer, `infra/monitoring`
Interview media	Session tokens/signaling; browser client	`LiveKitTokenService`, `InterviewSignalingService`, LiveKit frontend package

Request flow

Backend layers

- `com.hrms.config`: security, CORS, WebSocket, async, rate limiting, request ID and data initialization.
- `com.hrms.modules`: domain modules; a module may not use every layer consistently.
- `com.hrms.entity` and `com.hrms.repository`: shared/core domain model and repositories.

- `com.hrms.dto`: core request and response transport objects.
- `com.hrms.security` and `infrastructure`: JWT, tenant context, Redis blacklist and storage implementation.
- `com.hrms.exception`: global error mapping and domain exception types.
- `com.hrms.common`: audit and mail support.

Cross-cutting behavior and dependencies

The tenant filter establishes request scope; JWT claims may also set tenant context. Modules depend on shared employee/user/department records. Leave, attendance regularization and employee requests can use `WorkflowEngineService`; notifications and audit are shared services. Payroll references employee/attendance/expense data. Offboarding references employee records and settlement data. Recruitment conversion creates employee/user-related records (verify exact transactional semantics in `RecruitmentService`).

Integration boundaries

Confirmed repository integrations are SMTP email, Redis, local filesystem upload storage, Prometheus scraping, and interview signaling/media token code. Docker Compose provisions PostgreSQL and Redis. No confirmed live bank, biometric-vendor, external identity, or AI-provider client is established from the reviewed integration code. Biometric input is represented by REST/CSV ingestion endpoints and device metadata.

Architecture risks / verification

- Deployment profile and actual DB schema must be reconciled with V1–V15 migrations.
- The JWT and tenant filters' ordering and tenant claim/header precedence require security testing in the deployed configuration.
- `LocalStorageService` implies node-local files unless the deployment mounts shared persistent storage.
- WebSocket configuration is present; verify frontend subscriptions and authorization before relying on push delivery.
- A large shared application and domain service surface means regression and tenant-isolation testing are important; this document does not certify them.

Module Wise Documentation

Status labels describe evidence in source: **Implemented surface** means controller/service/UI or persistence code exists; it does not assert complete production behavior. **Partial / verify** identifies gaps or unverified paths.

Module	Backend implementation	UI/API surface	Dependencies and status
Authentication	`modules/auth/AuthController`, `AuthService`, `TotpService`; shared JWT/security	Login, register, reset, verify and first-login pages	Implemented surface; logout-all behavior incomplete; see security doc
Employee & profile	`EmployeeController`, `EmployeeService`, `Employee`, `EmployeeRequest/Response`	Directory, form, details, profile	Implemented; ownership/role enforcement requires endpoint-level review
Departments	`DepartmentController`, `DepartmentService`, `Department`	Department page	Implemented CRUD surface
Attendance & policy	`AttendanceController`, `AttendanceService`, `GeofenceValidator`, policy resolver, regularization/device/location entities	Attendance page and Policy Config	Implemented surface; external biometric integration is ingestion, not vendor connector
Leave	`LeaveController`, `LeavePolicyController`, `LeaveService`, accrual scheduler	Leave page	Implemented surface; policy rules and scheduled execution need operational validation
Requests & workflows	`EmployeeRequestController`, `WorkflowController`, `WorkflowEngineService`, workflow entities	Requests and approvals pages	Implemented surface; approver filtering/notifications and SLA enforcement require verification
Payroll	`PayrollController`, `PayrollService`, `PayrollSlip`	Payroll page; PDF utility	Implemented processing/export surface; statutory formulas require business validation
Recruitment & interviews	Recruitment/Interview controllers and services, job/candidate/session/panelist/scorecard entities	Recruitment, guest lobby, interview room	Implemented surface; external media provider configuration and end-to-end signaling require verification

Module	Backend implementation	UI/API surface	Dependencies and status
Performance	`PerformanceController`, `PerformanceService`, goals/cycles/reviews	Performance page	Implemented surface
Expense	`ExpenseController`, `ExpenseServiceImpl`, `ExpenseClaim`	Expense claims	Implemented submit/review surface; payroll bridge should be verified in transactional flow
Documents & policy acknowledgement	`DocumentController`, `DocumentService`, expiry scheduler	Document Vault, document APIs	Implemented upload/download/verification/acknowledge surface; storage encryption not established
Offboarding	`OffboardingController`, `OffboardingService`, separation/clearance/settlement entities	Offboarding page	Implemented workflow surface; legal formulas and completion checks need validation
Shift & roster	`ShiftRosterController`, `ShiftRosterService`, shift/assignment/swap entities	Shift roster page	Implemented surface
Assets	`AssetController`, `AssetService`, asset/allocation entities	Asset page	Implemented allocation/return surface
Helpdesk	`HelpdeskController`, `HelpdeskService`, ticket/comment entities	Helpdesk page	Implemented ticket/comment/stats surface
Communication	`CommunicationController`, `CommunicationService`, announcement entities	Announcements page	Implemented announcement/acknowledgement surface
Notifications	`NotificationController`, `NotificationService`, shared notification entity	Notifications page	REST and WebSocket support exist; delivery semantics should be verified
Workforce intelligence	`WorkforceIntelligenceController`, service implementation, risk/policy entities	Workforce Intelligence page	Local scoring/assistant functionality exists; external generative AI integration unverified
Reports & dashboard	`ReportController`, `DashboardController`, services	Reports and Dashboard pages	Implemented query surface; metric definitions/export scope require business confirmation
Tenant & admin	tenant/admin/module controllers and services	Admin and policies pages	Implemented surface; ensure all privileged operations enforce tenant scope

Cross-module dependencies

Present but incomplete or not demonstrated

The current UI route list does not expose every API capability as a dedicated screen. The database migration directory does not visibly define every table represented by entities/seed data (notably core tables may be created in earlier/base SQL, and `tax_declarations` appears in V4). Confirm `docs/schema.sql`, seed SQL, and the deployed database as one source of truth. No separate time tracking, benefits enrollment, training/LMS, or expense attachment-specific receipt pipeline should be inferred unless the corresponding source path is confirmed.

Database Design

Persistence approach

The backend uses Spring Data JPA entities with PostgreSQL-oriented application configuration. Flyway scripts are located under `backend/src/main/resources/db/migration` and currently range from V1 to V15. `docs/schema.sql` and root `hrms_db_seed.sql` are supplementary artifacts; they may not match the current migration chain. Always validate against the schema deployed for a specific environment.

Entity/table inventory

Table names below come from entity `@Table` mappings. A listed entity does not prove its migration has been applied. Core tables without an explicit migration `CREATE TABLE` may be supplied by `docs/schema.sql` or earlier baseline history.

Area	Tables / entities
Identity and tenancy	`users`, `tenants`, `tenant_subscriptions`, `tenant_settings`, `tenant_module_manifests`, `roles`, `permissions`,

Area	Tables / entities
	role_permissions, user_roles, refresh_tokens, login_history
Organization	employees, departments, system_settings, audit_logs
Attendance	attendance, attendance_regularizations, biometric_devices, office_locations, attendance_policies, policy_assignments
Leave and requests	leave_requests, leave_policies, employee_leave_balances, employee_requests
Workflow	workflow_definitions, workflow_steps, workflow_instances, workflow_approvals, approval_delegations
Payroll and tax	payroll_slips, tax_declarations
Recruiting	job_postings, candidates, interview_sessions, interview_panelists, interview_scorecards
Performance	performance_reviews, appraisal_cycles, employee_goals
Documents	employee_documents, policy_acknowledgements
Assets and expenses	assets, asset_allocations, expense_claims
Operations/communications	notifications, support_tickets, ticket_comments, company_announcements, announcement_acknowledgments, shifts, roster_assignments, shift_swap_requests
Offboarding/intelligence	employee_separations, separation_clearances, full_and_final_settlements, hr_policy_articles, flight_risk_evaluations

Additional entity classes such as `Permission`/`TenantRole` should be cross-checked against pluralized join-table mappings and `docs/schema.sql`; naming is not inferred where no explicit mapping exists.

Relationship model (logical)

The diagram is conceptual. Confirm exact foreign-key constraints, nullable relationships and cascade behavior in entity annotations and SQL before applying data changes.

Tenant and audit model

`BaseTenantEntity` supplies tenant scope to tenant-aware entities; `BaseEntity` supplies common identity/audit fields where inherited. Migration V2 adds `tenant_id` to core tables and tenant-oriented indexes. V3–V15 add module tables/indexes. `TenantContextFilter` resolves a tenant per HTTP request. Tenant-aware repository methods are used in many modules, but object-level and query-level isolation should be reviewed per service.

Migration map

Version	Scope indicated by filename/content
V1	Payroll column/index repair; SQL syntax is MySQL-like and conflicts with PostgreSQL target
V2	SaaS tenancy/RBAC tables and tenant columns/indexes
V3	Locations, attendance regularizations, leave policies/balances, requests, workflow, documents
V4	Appraisal cycles/goals, scorecards, assets/allocations, tax declarations
V5	First-login password flag
V6	Attendance punch metadata and biometric devices
V7	Offboarding, clearance and settlement
V8	Helpdesk and communication
V9	Shift and roster management
V10	Workforce intelligence and HR policy content
V11	Expense claims and payroll reimbursement column
V12	Payroll draft/review adjustments/status/index
V13	Interview sessions, guest access and scorecard updates
V14	Multi-participant interview panelists
V15	Tenant module manifests, attendance policies/assignments, approval delegations

Database concerns

- Run a migration compatibility review before applying the full chain to PostgreSQL; specifically inspect V1 and version ordering/baseline state.

- Reconcile JPA `ddl-auto`, Flyway state, `docs/schema.sql`, `hrms_db_seed.sql`, and environment schema. Avoid treating seed SQL as migration evidence.
- Document retention, PII classification, backup/restore, encryption, indexes and tenant deletion policies with operations and data owners.

API Documentation

Conventions

Controller mappings use `/v1` paths. The default `application.properties` sets `server.servlet.context-path=/api`, so deployed/local default paths are `/api/v1/...`; a different active profile or deployment override may change this. Endpoints generally wrap JSON results in `ApiResponse<T>`. Standard request bodies use Jakarta validation where annotated. Pagination uses Spring `Page<T>` on selected list operations. Upload routes consume `multipart/form-data`; document download returns a resource. Authentication is usually `Authorization: Bearer <access-token>`. Tenant context can be supplied with `X-Tenant-ID`; details are in the security document. Exact response fields should be taken from each DTO, not inferred from route names.

The API inventory below reflects controller mappings. Authorization can additionally come from `SecurityConfig`, class/method `@PreAuthorize`, or service checks; missing annotations do not mean public access because most paths require authentication.

Common payloads confirmed in source

`ApiResponse<T>` serializes non-null fields `success` (boolean), `message`, `data`, `timestamp`, `requestId`, and `statusCode`. Success factory methods populate `success/message/data`; error response shape/status is controlled by `GlobalExceptionHandler` and can omit null members.

`POST /v1/auth/login` accepts `LoginRequest`: `email` (required email), `password` (required), `rememberMe` (Boolean, default false), and optional `totpCode` (Integer). Its `AuthResponse` includes `accessToken`, `refreshToken`, `tokenType` (default `Bearer`), `tenantId`, `userId`, `employeeId`, `email`, `fullName`, `role`, `emailVerified`, `profilePicture`, `expiresIn`, `totpEnabled`, and `mustChangePassword`. The controller also sets the refresh cookie.

`POST /v1/attendance/punch` accepts `AttendancePunchRequest`: required `employeeId`, `date`, `time`, `punchType`; optional `latitude`, `longitude`, `ipAddress`, `officeLocationId`, `selfieUrl`, `remarks`, `lateArrivalReason`, `punchSource`, `deviceId`, and `workMode`. The response data type is `AttendanceResponse`.

`POST /v1/workflows/tasks/{instanceId}/decision` accepts `ApprovalActionRequest` with required `decision` and optional `comments`, and returns `WorkflowTaskResponse`. Although the DTO comment lists `REQUEST_CHANGE`, `WorkflowEngineService.processApproval` currently accepts only `APPROVED` and `REJECTED` and throws for other values.

`POST /v1/requests` uses `EmployeeRequestCreateDto`: required `employeeId`, `requestType`, `title`, optional `description`, `startDate`, `endDate`, `amount`, and `attachmentUrl`. The accepted request types and constraints should be confirmed in `EmployeeRequestService`; the DTO comment lists WORK_FROM_HOME, SHIFT_CHANGE, EXPENSE_CLAIM, TRAVEL_REQUEST, and BONAFIDE_LETTER.

Example envelope (illustrative field values only):

```
{
  "success": true,
  "message": "Success",
  "data": {},
  "timestamp": "<server-local-date-time>",
  "requestId": "<request-id>",
  "statusCode": 200
}
```

The sample is a shape illustration; timestamps and optional properties depend on runtime serialization and request filters.

Endpoint inventory

Area	Method and path	Purpose / request shape
Auth	`POST /v1/auth/register`	`RegisterRequest`; returns `AuthResponse`
Auth	`POST /v1/auth/login`	`LoginRequest`; returns access/refresh data and sets refresh cookie
Auth	`POST /v1/auth/refresh`	refresh token in body or cookie; rotates token
Auth	`POST /v1/auth/logout`, `/logout-all`	Revoke token or request session invalidation;

Area	Method and path	Purpose / request shape
		logout-all controller is currently a success stub
Auth	`POST /v1/auth/forgot-password`, `/reset-password`	OTP reset requests
Auth	`POST /v1/auth/verify-email`, `/resend-verification`	Query parameters `token/email` or `email`
Auth	`POST /v1/auth/change-password`	`ChangePasswordRequest`
Auth	`GET /v1/auth/login-history?userId=…`	Login history query
Auth	`POST /v1/auth/2fa/setup`, `/2fa/enable`, `/2fa/disable`	TOTP setup/verification; see controller DTOs
Employees	`GET /v1/employees`, `/me`, `/{id}`, `/department/{deptId}`	Employee lists/detail/profile; list endpoints accept pagination/filter parameters in controller
Employees	`POST /v1/employees`, `PUT /{id}`, `DELETE /{id}`	Create/update/soft-delete using `EmployeeRequest`
Employees	`POST /v1/employees/photo`, `/{id}/photo`, `/{id}/document`	Multipart upload
Departments	`GET /v1/departments`, `/{id}`; `POST /`, `PUT /{id}`, `DELETE /{id}`	Department CRUD with `DepartmentRequest`
Attendance	`POST /v1/attendance/punch`, `/auto-login-punch`, `/break-toggle`	Punch or toggle break; request DTO/employee ID per controller
Attendance	`GET /v1/attendance/today-status`, `/employee/{id}`, `/employee/{id}/monthly`, `/date`	Attendance status/list queries
Attendance	`POST /v1/attendance`, `PUT /{id}`	Administrative/manual record create/update
Attendance	`POST /biometric-sync`, `/biometric/simulate-punch`, `/biometric-upload`	Ingest JSON punches or CSV multipart; upload path is not proof of vendor connectivity
Attendance	`GET/POST /biometric-devices`, `/office-locations`	Device/location management
Attendance	`POST /regularize`, `PUT /regularize/{id}/review`	Request/review regularization
Policies	`GET /v1/policies/attendance`, `/attendance/{id}`, `/attendance/effective/{employeeId}`	List/detail/effective attendance policy
Policies	`POST/PUT/DELETE /v1/policies/attendance[/{id}]`; `GET/POST /assignments`; `DELETE /assignments/{id}`	Policy definition and assignment management
Leave	`POST /v1/leaves/apply/{empId}`, `PUT /{id}/review`, `PUT /{id}/cancel/{empId}`	Apply, review, cancel leave
Leave	`GET /v1/leaves/employee/{empId}`, `/pending`, `/policies`, `/balances/employee/{empId}`	Leave, policy and balance queries
Leave	`POST /v1/leaves/policies`, `/accrual/run`	Configure policy and trigger accrual
Requests	`POST /v1/requests`, `GET /employee/{empId}`, `/pending`, `PUT /{id}/review`	Employee request lifecycle
Workflows	`GET /v1/workflows/inbox`, `/definitions`, `/delegations`; `POST /tasks/{instanceId}/decision`, `/delegations`; `DELETE /delegations/{id}`	Workflow inbox, decisions, definitions and delegations
Payroll	`POST /v1/payroll/process`, `/process-all`, `/{id}/approve`, `/approve-all`, `/mark-paid`	Payroll processing/review lifecycle
Payroll	`GET /register`, `/summary`, `/bank-batch-export`, `/bank-batch`, `/employee/{empId}`, `/employee/{empId}/slip`; `PUT /{id}/adjust`	Payroll register, summary, exports and employee slips
Recruitment	`GET/POST /v1/recruitment/jobs`, `/candidates`; `PATCH /candidates/{id}/stage`	Job and candidate management
Recruitment	`POST/GET /candidates/{id}/scorecards`; `POST /candidates/{id}/convert-to-employee`	Candidate evaluation/conversion
Interviews	`POST/GET /v1/recruitment/interviews`, `GET /candidate/{candidateId}`, `/{id}`, `/{id}/join-token`	Session lifecycle and join token
Interviews	`GET /guest/{guestToken}`, `POST /guest/{guestToken}/join`, `/telemetry`, `/signal`; `GET /guest/{guestToken}/signals`	Guest access/signaling endpoints
Interviews	`PUT /{id}/scorecard`; `GET/POST /{id}/panelists`; `DELETE`	Interview evaluation, panel and signaling

Area	Method and path	Purpose / request shape
	`/{id}/panelists/{panelistId}`; `POST /{id}/complete`, `/cancel`, `/signal`; `GET /{id}/signals`; `GET /{id}/scorecards`	
Performance	`GET/POST /v1/performance/cycles`; `GET/POST /goals/employee/{empId}`, `/goals`; `PATCH /goals/{id}/progress`; `GET /reviews`, `/reviews/employee/{empId}`; `POST /reviews`	Cycles, goals and reviews
Expenses	`POST /v1/expenses/claims`; `GET /claims/my`, `/claims`, `/claims/{id}`; `PUT /claims/{id}/manager-review`, `/finance-review`	Expense submission, queries and staged review
Documents	`POST /v1/documents/upload`; `GET /employee/{empId}`, `/pending-verification`, `/{id}/download`; `PUT /{id}/verify`; `POST /{id}/acknowledge`, `/letters/generate`	Upload, verification, download, acknowledgement and letter generation
Offboarding	`POST/GET /v1/offboarding/separations`, `GET /separations/{id}`, `/separations/my`; `PUT /separations/{id}/review`, `/clearances/{id}`; `POST /settlements/calculate`, `/settlements`; `GET /settlements/separation/{separationId}`	Separation, clearances and settlement
Shifts	`GET/POST /v1/shifts`; `POST /roster/assign`; `GET /roster`, `/roster/my`; `POST /swaps`; incoming/outgoing/pending-manager swap GETs; peer and manager review PUTs	Shift, roster and swap lifecycle
Assets	`GET/POST /v1/assets`; `POST /{id}/allocate`, `/allocations/{id}/return`; `GET /employee/{empId}`	Asset inventory and allocations
Helpdesk	`POST/GET /v1/helpdesk/tickets`, `/tickets/my`, `/{id}`, `/stats`; `PUT /tickets/{id}`; `POST /tickets/{id}/comments`	Ticket creation, review, comments, stats
Communication	`POST/GET /v1/communication/announcements`, `GET /announcements/all`, `POST /announcements/{id}/acknowledge`, `DELETE /announcements/{id}`	Announcement lifecycle
Notifications	`GET /v1/notifications/user/{userId}`, `/unread-count`; `PUT /user/{userId}/mark-all-read`, `/{id}/read`	Notification queries/read markers
Intelligence	`POST /v1/intelligence/assistant/ask`; policy GET/POST/upload/PUT/DELETE paths; risk-analysis, re-evaluate and department-overview; `POST /ats/evaluate-candidate`	Assistant/policy/risk/candidate matching APIs
Reports/dashboard	`GET /v1/reports/analytics`, `/tax/employee/{empId}`; `POST /tax`; `GET /v1/dashboard/admin`	Analytics, declarations and dashboard
Admin	`GET /v1/admin/users`, `/audit`, `/settings`; `PATCH /users/{id}/status`, `/users/{id}/role`; `PUT /settings/{key}`	User, audit and system settings
Tenants	`GET /v1/tenants`, `/current`, `/{id}`, `/{id}/settings`; `POST /`; `PUT /{id}/settings/{key}`, `/current/branding`, `/{id}/branding`; logo multipart POST paths	Tenant management and branding
Tenant modules	`GET /v1/tenant/modules`, `/enabled`; `PUT /{moduleKey}`, `/batch`	Manifest reads/toggle/batch update

Response and error handling

Most endpoints return `ApiResponse` with success/message/data fields (verify `ApiResponse.java` for exact null/error conventions). Exceptions are mapped by `GlobalExceptionHandler`; likely domain categories include bad request, not found, duplicate resource and token refresh errors. Do not assume one status code for every endpoint without checking handler mappings.

Interactive reference

SpringDoc/OpenAPI dependencies and Swagger paths are configured in `application.properties` and `SecurityConfig`. Run the backend and inspect the configured `/swagger-ui` and `/v3/api-docs` paths for generated request schemas. Runtime availability depends on active profile and security configuration.

Authentication and Authorization

Authentication flow

`SecurityConfig` configures a stateless `SecurityFilterChain`. `AuthController` delegates to `AuthService`; `JwtAuthenticationFilter` reads `Authorization: Bearer …`, extracts username/tenant, checks blacklist status, loads `UserDetails`, validates the token and establishes Spring Security authentication. Password encoding uses `BCryptPasswordEncoder(12)`.

The refresh endpoint accepts a body token or cookie and issues a replacement cookie/token response. Login sets `refreshToken` as HttpOnly, SameSite=Lax, path `/`, max age seven days; the code currently sets `secure(false)`, which requires production review. Frontend code also reads a refresh token from local storage in `App.tsx`; verify which token storage path is active and avoid assuming cookie-only handling.

Account lifecycle controls

Auth DTOs and services support registration, email verification/resend, login history, forgot-password OTP/reset, password change and first-login password status. TOTP setup/enable/disable endpoints and `TotpService` exist. The exact enforcement of TOTP at login, recovery-code behavior, lockout thresholds, OTP expiry and resend throttling must be confirmed in `AuthService`/configuration; endpoint presence alone does not prove each control is activated.

Tenant resolution

`TenantContextFilter` reads `X-Tenant-ID`, alternate casing, `tenantId` query parameter, then host subdomain and finally default tenant. It writes the resolved ID in the response header and clears thread-local context in `finally`. JWT processing may replace the context from a token tenant claim. Tenant context should match the authenticated user's tenant; confirm mismatch handling and all repository predicates.

Roles and route protection

`Role.java` defines role constants. The source includes `ROLE_SUPER_ADMIN`, `ROLE_ADMIN`, `ROLE_HR`, `ROLE_MANAGER`, `ROLE_EMPLOYEE`, `ROLE_IT_ADMIN`, and `ROLE_IT_SUPPORT` usage. `SecurityConfig` explicitly permits `/v1/auth/**`, current tenant metadata, guest interview paths, Swagger, selected actuator paths and `/uploads/**`; admin, dashboard, employee and department paths receive route-level restrictions. Most other APIs require authentication, with additional controller method-level `@PreAuthorize` rules.

Role	Examples of guarded capabilities (not exhaustive)
`ROLE_SUPER_ADMIN`	Tenant creation/listing, selected tenant/admin, recruitment and system operations
`ROLE_ADMIN`	Employee/department administration, payroll/tenant settings, policy and operational configuration
`ROLE_HR`	Employee, leave, attendance, payroll, recruitment, documents and policy workflows
`ROLE_MANAGER`	Selected leave, regularization, shift, workflow and team review paths
`ROLE_EMPLOYEE`	Authenticated self-service routes, workflow inbox and request flows
`ROLE_IT_ADMIN`	Selected admin/dashboard, assets, helpdesk and employee route access
`ROLE_IT_SUPPORT`	Helpdesk update path and IT-oriented support behavior

This is not a complete permission matrix. A number of endpoints accept user/employee IDs from request paths or query parameters and do not show an explicit `@PreAuthorize` annotation in the controller; verify ownership checks in service code and database queries.

Security controls and limitations

- `JwtAuthenticationFilter` checks a distributed blacklist service; Redis configuration and fallback behavior must be confirmed for production.
- `RateLimitConfig` and `CorsConfig` exist; actual per-route policy/origin values vary by active config and are not reproduced here.
- CSRF is disabled because the API is stateless; refresh cookie use still warrants review of cross-site request behavior.
- The logout-all route currently returns a success response without visible service invalidation in its controller.
- Public `/uploads/**` access is configured. Assess whether user documents or private files can be exposed through this route.
- API authorization does not, by itself, prove tenant isolation; review entity scope, services and repositories together.

User Workflows

Flows below reflect controller/service behavior and UI route surfaces. Exact validation, statuses and notification behavior can differ by module.

Sign in and password recovery

1. User enters credentials on `/login`; frontend calls `authAPI`.
2. `AuthService.login` validates credentials and returns `AuthResponse`; controller sets refresh cookie.
3. Protected route access uses the access token. On expiry, the client can call refresh.
4. Users can verify email, request a reset OTP and submit reset credentials via the corresponding auth pages.
5. Users flagged `mustChangePassword` are routed to `/first-login-change-password`.

Employee self-service and HR administration

1. HR/Admin creates an employee via `POST /v1/employees` and `EmployeeService` persists associated records.
2. User can view profile and upload a photo/document through employee APIs.
3. HR updates employee fields; deletion follows service soft-delete behavior (confirm retention rules).
4. Department assignment ties employee records to organizational grouping.

Attendance and regularization

The service exposes web/auto punch, break tracking, biometric JSON/CSV ingestion, geofence validation, policy resolution and regularization. Approval can be routed through workflow support, but exact workflow activation is dependent on configured workflow definitions.

Leave request and approval

1. Employee submits a leave application with employee ID and request details.
2. Service validates request and creates a pending leave record.
3. A manager/HR reviewer or workflow decision approves/rejects.
4. On workflow completion, `WorkflowEngineService` changes leave status and, for approval, updates a matching employee leave balance when found.
5. Notification is sent to employee. Cancellation and policy/balance endpoints support follow-up.

General employee request and approval

1. Employee creates an `EmployeeRequest` for a supported request type.
2. Review or workflow processing records decisions and comments.

3. On completion, request status is updated and notification is sent. Delegation records can be created for out-of-office coverage.

The code does not establish the README-claimed universal request types, atomic conflict checks or SLA deadlines as guaranteed behavior; confirm `EmployeeRequestService`, workflow configuration and database constraints.

Payroll processing

1. Authorized payroll user initiates a single or batch process.

2. `PayrollService` gathers employee and attendance data and calculates payroll records.

3. HR can adjust/approve records; approved payroll can be marked paid.

4. Payroll register, summaries, employee payslips and bank batch exports are exposed.

5. Frontend includes payslip PDF generation utility.

Formula correctness, tax configuration, expense inclusion and bank file acceptance require payroll-owner confirmation.

Recruitment and interview

1. Recruiter creates a job and candidate record.

2. Candidate stage and scorecards are updated through recruiting APIs.

3. Interview session is scheduled; a guest token can be used by a candidate to join a lobby/room.

4. Interviewers can join, exchange signaling messages and submit scorecards; panelists may be assigned.

5. Candidate conversion endpoint supports converting a candidate into employee onboarding. Verify atomicity, generated credentials, leave seeding and email behavior in service implementation.

Expense, helpdesk and communications

- Expense: employee submits a claim; manager review precedes finance review endpoints. The implemented enum/status transitions should be confirmed in `ExpenseServiceImpl`.
- Helpdesk: user creates ticket, comments may be added, support roles update ticket, and stats are queried.
- Announcements: authorized staff creates/deletes announcements; active/all lists and acknowledgement endpoint support employee response.

Offboarding

1. Employee submits separation request.

2. Authorized reviewer reviews it and clearance records are updated.

3. Settlement is calculated/saved and queried by separation.

4. Final employee status changes and generated documents depend on service behavior; verify required clearance completeness before relying on termination finalization.

UI/UX Documentation

Application shell and navigation

`frontend/src/App.tsx` defines public auth routes, guest interview routes, protected interview routes and the authenticated `MainLayout`. `MainLayout`, `Sidebar` and `Topbar` provide the main navigation shell. `ProtectedRoute.tsx` checks authentication and supports optional role allow-lists. `tenantSlice` loads tenant branding and module state influences navigation where integrated.

Route inventory

Route	Screen
`/login`, `/register`, `/forgot-password`, `/reset-password`, `/verify-`	Authentication

Route	Screen
email	
/first-login-change-password	Mandatory first-login change
/interview/guest/:guestToken, /interview/guest/:guestToken/room	Candidate interview access
/recruitment/interview/:sessionId	Protected interviewer room
/dashboard, /approvals, /requests, /attendance, /shifts, /intelligence, /leaves	Operational and self-service workspace
/documents, /employees, /employees/new, /employees/:id, /employees/:id/edit, /departments	Employee and organization management
/payroll, /expenses, /notifications, /profile	Compensation and employee experience
/performance, /recruitment, /assets, /helpdesk, /announcements, /reports, /offboarding	Business modules
/policies, /admin	Restricted policy and administration screens

Frontend structure and patterns

- src/modules/<module>/pages contains feature screens; services often wraps module API calls.
- src/api contains API clients and module endpoints; src/shared/services/apiClient.js and src/api/apiClient.ts are shared client surfaces.
- Redux Toolkit store is in src/store; authSlice, tenantSlice and uiSlice are visible slices.
- TypeScript domain contracts live in src/types; codebase mixes .tsx/.ts and .jsx/.js.
- Tailwind CSS and shared components provide styling; Recharts is used for charting; jsPDF utilities create downloadable documents.
- Error boundary and common components exist under src/components/common and src/shared/components.

UX behavior evidenced by code

The app refreshes tenant branding on startup and attempts access-token refresh when a refresh token is found in local storage. Authenticated employees default to /profile; users requiring a password change are sent to the change page; others default to /dashboard. Candidate guest routes are intentionally accessible without the normal authenticated shell. A fallback 404 screen exists.

Accessibility and responsive behavior

Tailwind-based responsive styling is present, but no verified accessibility audit, keyboard-navigation coverage, contrast validation, localization strategy, or formal design system specification was found. Confirm these with product/design owners. The UI includes profile-photo cropper and generated PDF utilities.

Installation and Setup

Prerequisites

- Java 21 (project README and compiler configuration target Java 21)
- Maven 3.9+ (wrapper availability is not established in the repository inventory)
- Node.js/npm compatible with the frontend lockfile and Vite toolchain
- PostgreSQL version supported by the chosen application profile (Compose provisions PostgreSQL 16)
- Optional Redis for token blacklist behavior
- Docker and Docker Compose for the containerized path

Configuration

Use .env.example as the variable-name reference. Do not commit real credentials. Backend configuration is in backend/src/main/resources/application*.properties; active profile determines datasource, JWT, mail, upload, CORS, rate limit and actuator settings. Secret values are intentionally excluded from this document. Verify required environment variable names and defaults in properties/Compose before deployment.

Local backend

1. Provision PostgreSQL and create an application database/user.
2. Configure environment values for datasource, JWT secret, mail, upload location and optional Redis.
3. From `backend`, build and run with Maven, selecting the intended Spring profile.
4. Confirm Flyway migration logs and health endpoint before using the API.

The repository includes `setup-postgres.bat`; inspect it before running because it is an environment-specific helper. Review migration V1 for PostgreSQL compatibility before first migration.

Local frontend

1. From `frontend`, run `npm ci` using the committed lockfile.
2. Set the API base URL and any tenant configuration expected by `vite.config.js` / `.env` files.
3. Run the Vite development server with the package script.
4. Open the served URL and sign in with an account provisioned for the selected tenant.

No default credentials are reproduced here. Review `DataInitializer` and seed SQL for development-only account setup, and change/remove any seeded credentials before use outside a disposable local environment.

Docker Compose

The root `docker-compose.yml` declares PostgreSQL, Redis, backend, frontend, Prometheus and Grafana, with named volumes for database, Redis, uploads, logs and Prometheus data. Use environment overrides from a secure local file and check service health before testing the UI. Ports and credentials are defined in configuration files; verify locally rather than copying values into documentation.

Profiles and checks

Backend profile files include `application.properties`, `application-pg17.properties`, `application-h2.properties`, plus `src/test/resources/application-test.properties`. Confirm the intended profile; H2 tests do not prove PostgreSQL migration compatibility. OpenAPI paths are configured through SpringDoc.

Testing Documentation

Test assets found

Location	Type / purpose
`backend/src/test/java/com/hrms/service/AuthServiceTest.java`	Auth service unit tests
`backend/src/test/java/com/hrms/controller/AuthControllerIntegrationTest.java`	Auth controller integration tests
`backend/src/test/resources/application-test.properties`	Test configuration
`frontend/e2e/login.spec.js`	Playwright login E2E test
`frontend/playwright.config.js`	Browser test configuration
Root `test_phase*.ps1`, `test_it_service_desk_and_communication.ps1`	PowerShell API/feature test scripts
`run_senior_qa_e2e_tests.ps1`	Broader QA runner
`scripts/e2e-suite.js`, `master-e2e-deep-test.js`, `test_policy_engine.cjs`	JavaScript E2E/policy test assets
`frontend/test-results/`	Existing Playwright artifacts; not a fresh test run

Coverage and limitations

Visible committed tests emphasize authentication and selected workflow/module E2E scenarios. The test file inventory does not establish coverage for every controller, authorization boundary, tenant isolation query, migration, browser flow or external integration. Documentation generation did not execute tests. Existing output files and README test-count claims are not treated as current test results.

Suggested verification areas

- Run Maven unit/integration tests with the intended PostgreSQL-compatible profile and validate full Flyway chain from a clean database.
- Run frontend build/typecheck and Playwright suite against a disposable configured environment.
- Add/confirm tests for role and ownership enforcement, cross-tenant IDOR attempts, JWT refresh/logout/replay, cookie flags, TOTP login enforcement, workflow transitions, payroll rounding, offboarding clearance gates, upload validation and storage access.
- Confirm database migration rollback/recovery strategy and concurrency tests for overlapping attendance/request and payroll processing operations.

These are recommendations only; no test or build was run for this documentation task.

Deployment and Maintenance

Deployment assets

The repository contains Dockerfiles for backend/frontend, root `docker-compose.yml`, frontend `vercel.json`, frontend and infrastructure Nginx configurations, GitHub Actions workflow files under `.github/workflows`, and Prometheus/Grafana configuration under `infra/monitoring`. Compose provisions PostgreSQL, Redis, API, frontend, Prometheus and Grafana.

Deployment outline

1. Build backend and frontend artifacts using locked dependency versions.

2. Provision PostgreSQL and Redis with persistent volumes and restricted network access.

3. Supply secrets through the hosting platform's secret manager; do not rely on checked-in defaults.

4. Configure datasource, JWT signing key, mail, CORS, frontend API URL, tenant host routing, upload volume and monitoring endpoints.

5. Deploy API and frontend behind TLS-terminating ingress/proxy; ensure refresh cookies use secure production attributes.

6. Review and apply Flyway migrations with a tested backup/restore plan; validate V1 SQL dialect compatibility.

7. Verify health/readiness, login, tenant resolution, file persistence, mail and monitoring after rollout.

The steps describe deployment dependencies, not a verified production runbook. No specific cloud platform or release approval procedure can be confirmed solely from repository artifacts.

Monitoring and maintenance

- Spring Actuator health/info and Micrometer Prometheus registry are configured.
- `infra/monitoring/prometheus.yml` configures scraping; Grafana datasource provisioning exists.
- Logging configuration writes application logs; Compose has a logs volume.
- Review retention and access to application logs, uploaded documents, monitoring dashboards and audit data.
- Maintain PostgreSQL and Redis backups, restore drills, disk/volume monitoring, TLS certificate rotation, dependency updates and security incident procedures.
- Keep Flyway migration state synchronized across environments; do not edit already-applied migrations.

Operational risks requiring confirmation

- Local storage must be persistent/shared if multiple backend instances serve uploaded files.
- Redis availability/fallback behavior affects revocation semantics.
- Rate limiting is described in code/config, but distributed enforcement across replicas is not established.
- CI workflow artifacts and status snippets in repository are not proof of current release status.
- Validate health endpoint exposure and whether actuator detail/metrics are adequately restricted.

- Establish backups, recovery time/objectives, supported upgrade window, data retention and tenant offboarding procedure with operations.

Project Structure

Repository root

Path	Responsibility
`backend/`	Spring Boot service, Maven build, Java source and resources
`frontend/`	React/Vite SPA, package lock, UI source, Playwright configuration/tests
`infra/`	Nginx and monitoring configuration
`docs/`	Existing schema, technical notes, diagrams, screenshots and presentation artifacts
`scripts/`	E2E, diagram/document generation and test utilities
`.github/workflows/`	CI/CD workflow definitions
`storage/`, `logs/`	Runtime-oriented storage/log paths in working tree; confirm ignored/tracked status before deployment
`docker-compose.yml`	Local/container service composition
`.env.example`	Environment variable names/template; contains no values copied into this package
`README.md`	Existing project guide (claims may be aspirational or stale; source code takes precedence)
`hrms_db_seed.sql`	Seed data/schema-related SQL artifact; not a migration substitute
`project-documentation/`	This source-grounded documentation package

Backend source

Path	Responsibility
`backend/src/main/java/com/hrms/modules/<module>/controller`	REST controllers and route mappings
`.../service`	Business logic and orchestration
`.../repository`	Spring Data JPA repository interfaces
`.../entity`	Persistent domain entities
`.../dto`	Transport/request/response contracts
`com/hrms/entity`, `repository`, `dto`	Shared/core domain classes
`com/hrms/config`	Spring security, CORS, filters, async, WebSocket, rate limits, initialization
`com/hrms/security`	JWT and user-details integration
`com/hrms/infrastructure`	Tenant context, Redis blacklist and local storage adapters
`com/hrms/common`	Shared audit and mail services
`backend/src/main/resources/db/migration`	Flyway SQL migrations V1–V15
`backend/src/main/resources/templates/email`	Thymeleaf email templates
`backend/src/test`	JUnit/Spring test code and test profiles

Frontend source

Path	Responsibility
`frontend/src/App.tsx`	Route composition and startup effects
`src/modules/<module>/pages`	Feature screens
`src/modules/<module>/services`	Feature-level API/service wrappers
`src/api`	API clients and endpoint helpers
`src/shared/services`	Shared API service client
`src/store`	Redux Toolkit store, slices and hooks
`src/components/layout`, `src/shared/layout`	Application shell components; note parallel/legacy-looking paths
`src/components/common`, `src/shared/components`	Shared widgets and error boundary
`src/types`	Frontend domain/API type definitions
`src/styles`	Global styles and Tailwind integration
`frontend/e2e`, `playwright.config.js`	Browser E2E test setup

Notable implementation observation

There are parallel `components/layout` and `shared/layout` directories and both TS/TSX and JS/JSX files. Confirm which duplicate-looking components are actively used before refactoring; this documentation task made no source changes.

HRMS Project Documentation

This documentation describes the implementation present in this repository at review time. Source code and the latest database migrations take precedence over older design documents and README feature claims. Items marked **partially implemented** or **verify** need operational or product confirmation.

Documents

1. Project overview
2. Software requirements specification
3. System architecture
4. Module wise documentation
5. Database design
6. API documentation
7. Authentication and authorization
8. User workflows
9. UI/UX documentation
10. Installation and setup
11. Testing documentation
12. Deployment and maintenance
13. Project structure

Evidence and scope

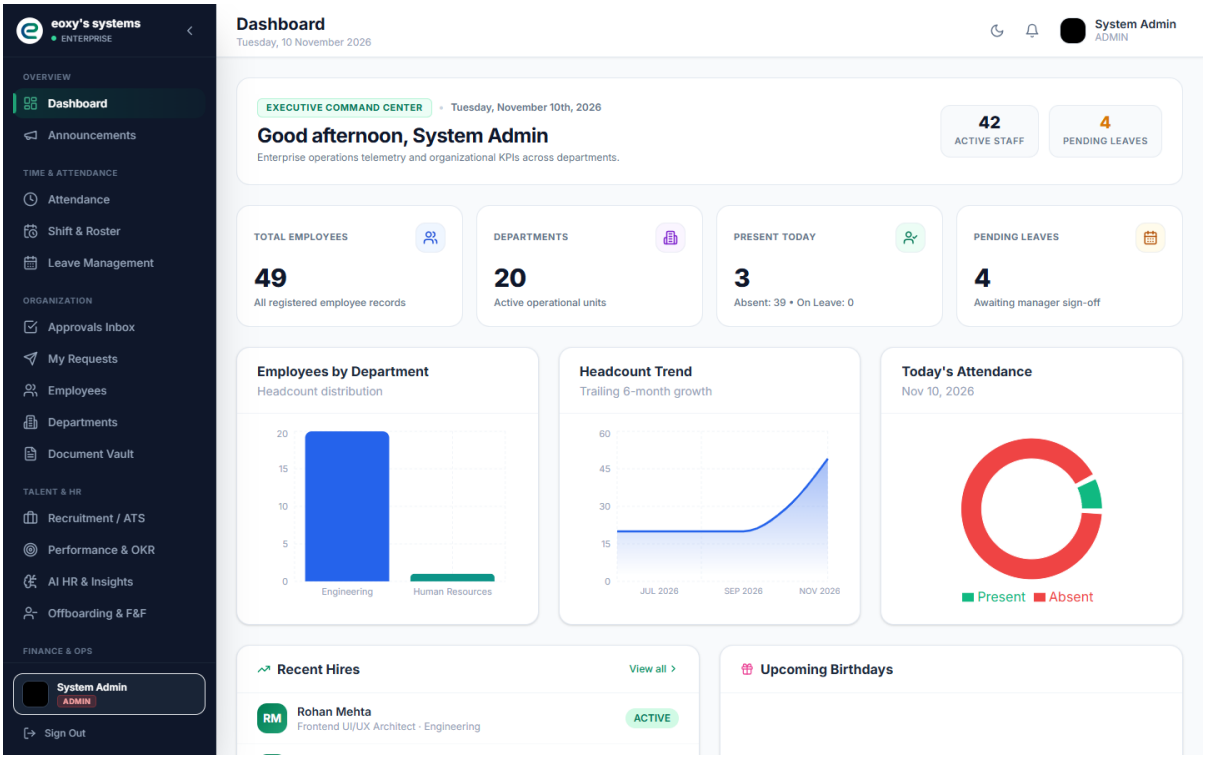
Primary evidence: `backend/src/main/java`, `backend/src/main/resources/db/migration`, `backend/src/test`, `frontend/src`, `frontend/package.json`, `docker-compose.yml`, Dockerfiles, CI workflow files and `.env.example`. No application source or existing documentation was changed to create this package. Secret values are intentionally omitted. Repository state contained pre-existing uncommitted changes; this package describes the working tree as inspected.

Known review items

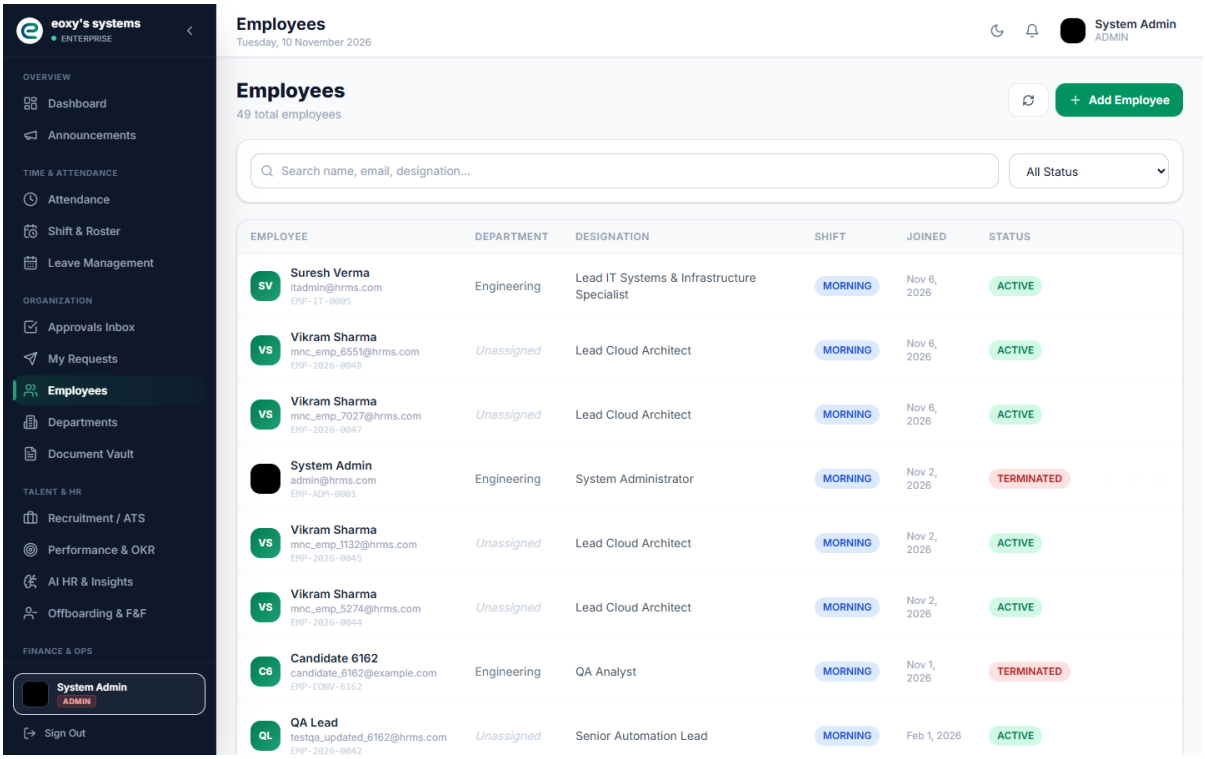
- Migration `V1__fix_payroll_columns.sql` uses MySQL-style `DROP INDEX` and `MODIFY COLUMN`, while runtime configuration and later migrations are PostgreSQL-oriented. Confirm the migration history that has actually been applied and whether V1 is compatible with the target database.
- Several controller routes identify records using supplied numeric IDs. Authorization annotations are not uniformly present on self-service endpoints; verify tenant and ownership enforcement in service methods before production exposure.
- `AuthController.logoutAll` currently returns success without revoking sessions in its controller implementation.
- The configured refresh cookie sets `secure(false)` in `AuthController`; confirm production cookie handling at the deployment boundary.
- Some modules have entities and UI/API surfaces but no repository migration in the visible migration directory, including recruitment/job posting data and tax declarations; reconcile these with the supplied SQL schema and deployed database.

Project UI Examples

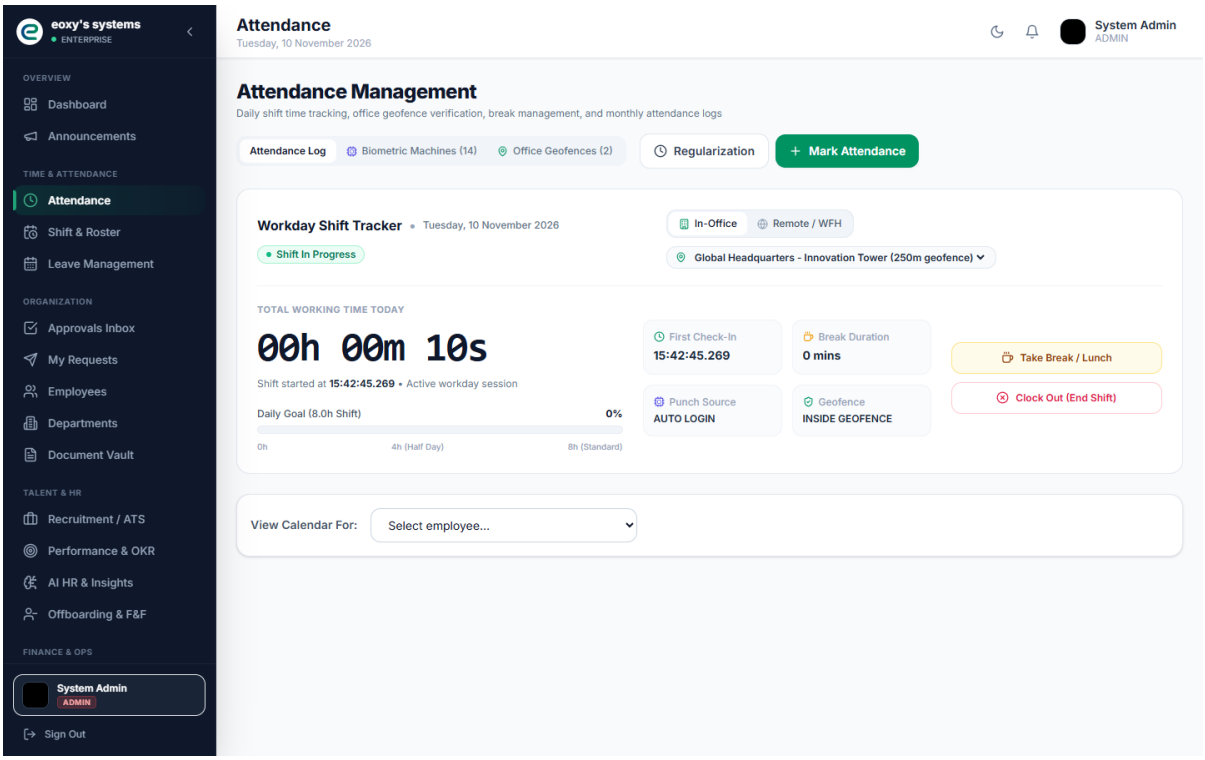
The following screenshots are existing repository captures. They illustrate implemented application surfaces and are not newly generated mockups.



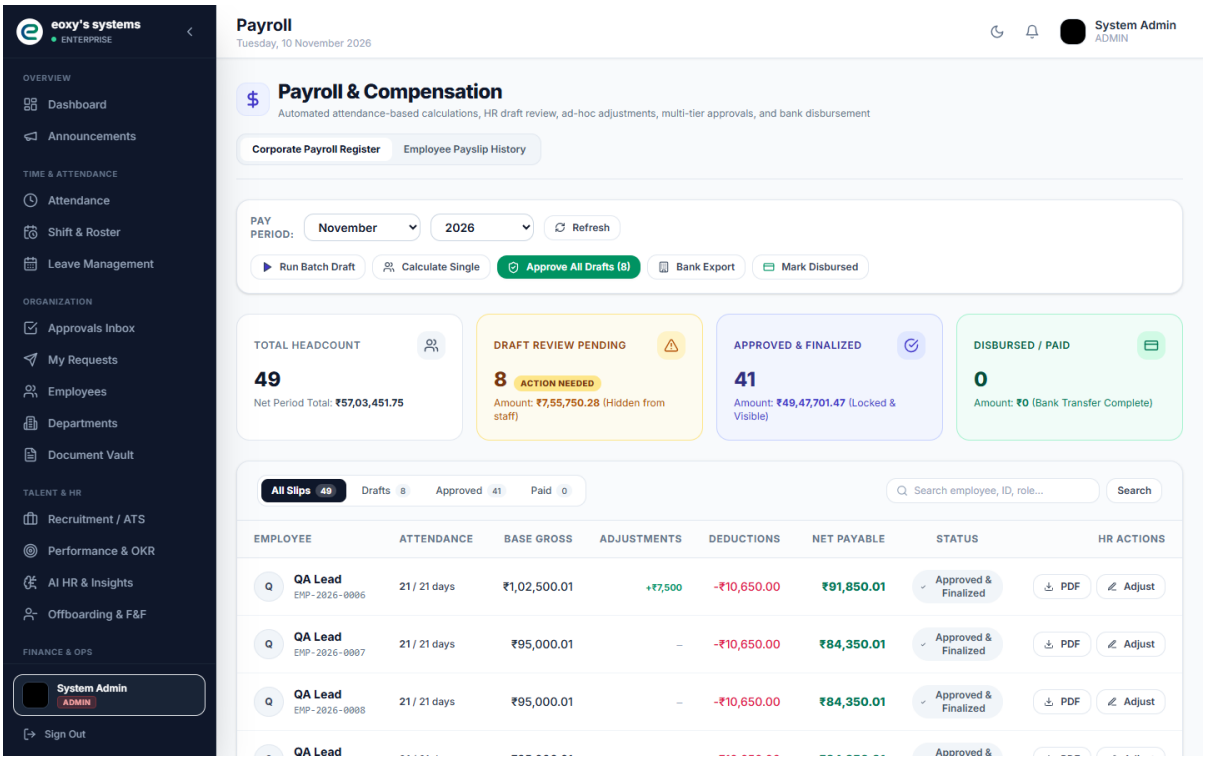
Dashboard overview



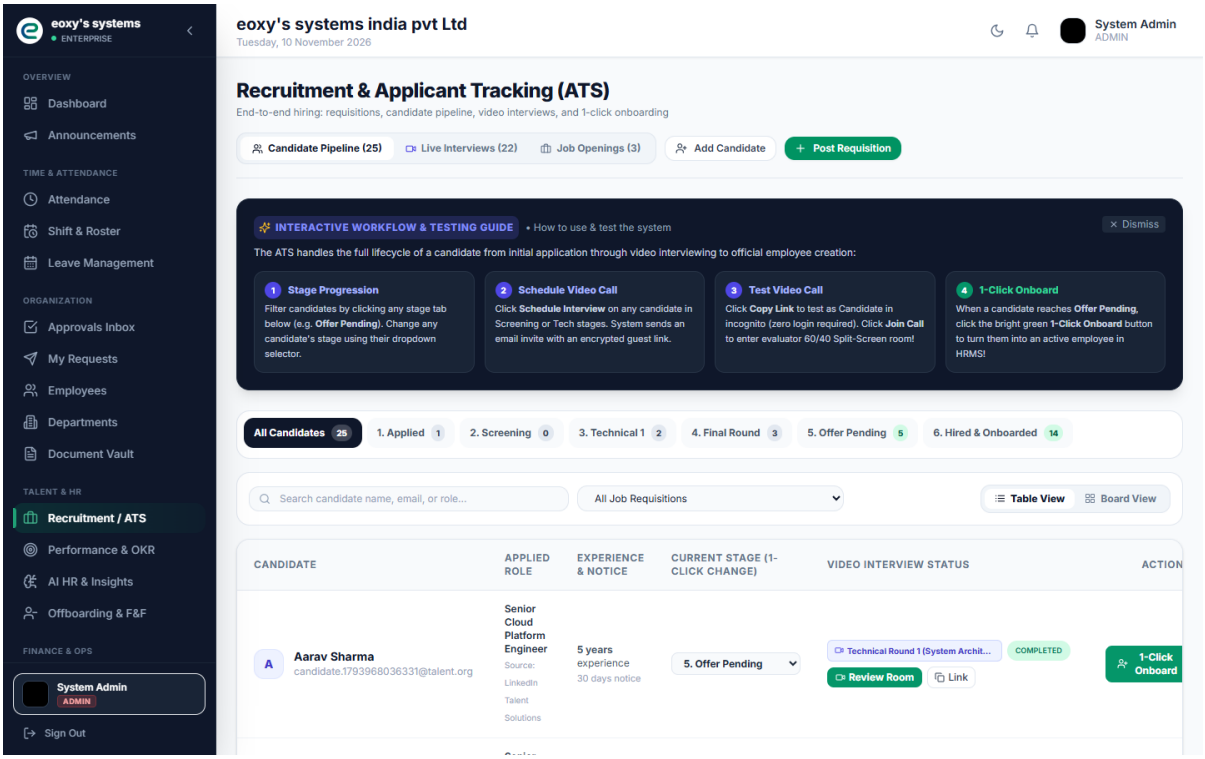
Employee directory



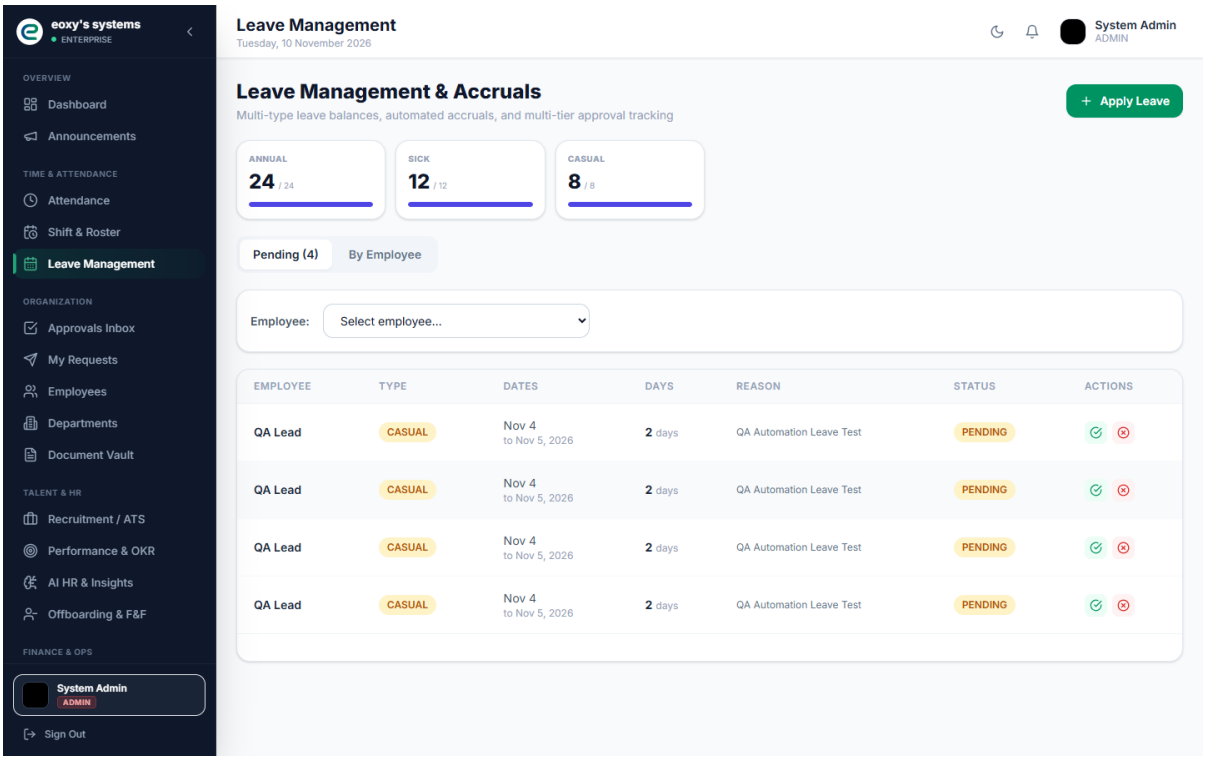
Attendance workspace



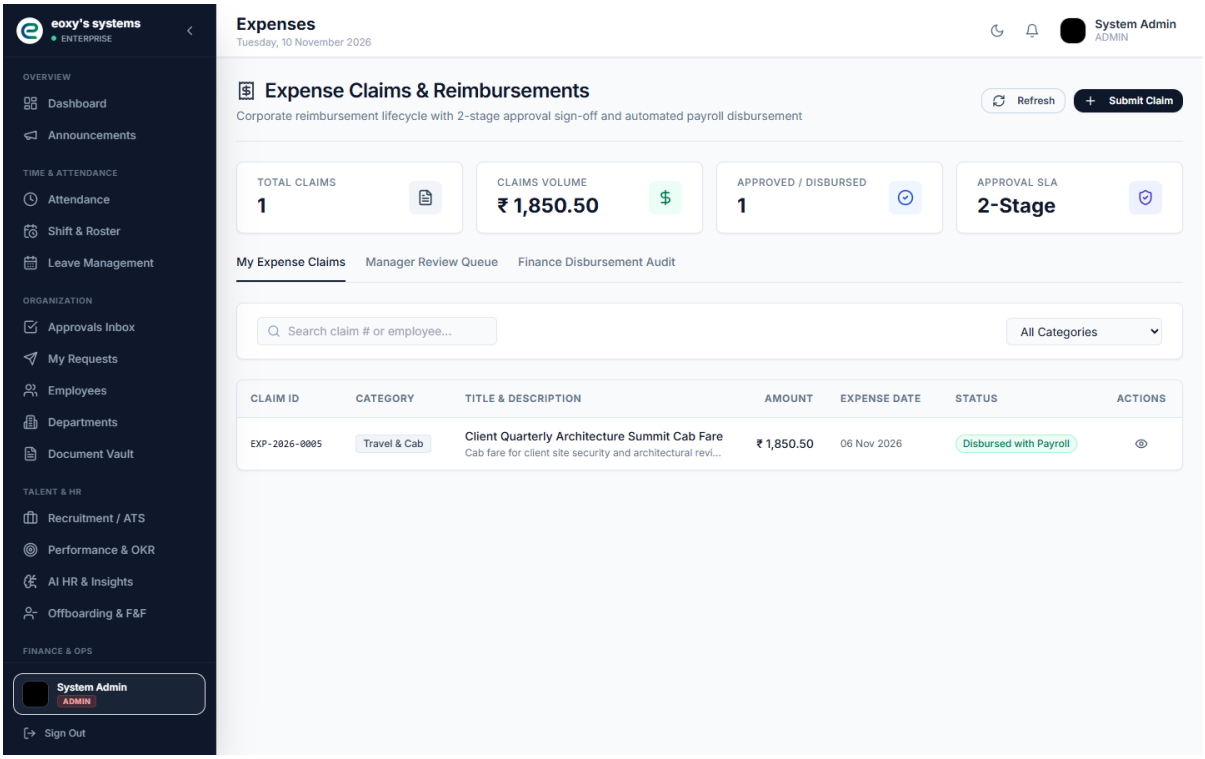
Payroll register



Recruitment pipeline



Leave management



Expense claims